

Installation

Documentation

Getting Started

Connect

Data Import and Export

Lakehouse Formats

Client APIs

SQL

Configuration

Extensions

Core Extensions

Quack Remote Protocol

Overview

Reference

Security

Setup

Troubleshooting

Guides

Operations Manual

Development

Internals

Sitemap

Live Demo

Documentation / Quack Remote Protocol

 Copy Markdown

Quack Remote Protocol

 We released Quack on May 12, 2026.
Read the [announcement blog post!](#)

The Quack extension turns a DuckDB instance into a server that other DuckDB instances (clients) can connect to over HTTP.

This page covers the protocol at a glance and walks through basic usage on both sides of the wire. For the full list of functions, settings, and logging knobs, see the [Reference](#). For configuring TLS and authentication / authorization, see [Security](#). For user guides, see [Guides](#).

 Quack is under active development and the protocol, function names, settings, and defaults are still subject to change. This page documents the beta release of Quack, available in [DuckDB v1.5.3](#), shipped via the `core` repository.

Quack in a Nutshell

In short, the Quack protocol and its interactions work as follows:

- **HTTP-based.** Quack messages travel over plain HTTP or HTTPS via a reverse proxy (see [Security](#)). This means standard load balancers, firewalls, and reverse proxies handle Quack traffic the same way they handle any other

IN
THIS
ARTICLE

Qu...

Ser...

S...

U...

S...

Cli...

S...

A...

A...

H...

N...

Installation

Documentation

Getting Started

Connect

Data Import and Export

Lakehouse Formats

Client APIs

SQL

Configuration

Extensions

Core Extensions

Quack Remote Protocol

Overview

Reference

Security

Setup

Troubleshooting

Guides

Operations Manual

Development

Internals

Sitemap

Live Demo

HTTP service. There is no custom wire transport to operate.

- **Client-driven request and response.**

Every interaction is initiated by the client. The server does not initiate interactions via pushing.

- **application/duckdb serialization.**

Requests and responses are encoded with DuckDB's internal serialization primitives (the same code path used by the Write-Ahead Log). This avoids round-tripping data through an interchange format and keeps complex types (nested, decimals, intervals, ...) lossless across the wire.

- **Single round-trip per query.** After the initial connection handshake, a query needs only one request-response pair. Large results stream back in chunks via follow-up `FETCH` requests, optionally parallelized on multiple threads.

- **Default port: 9494.** All URIs use the `quack:` scheme, e.g., `quack:hostname:port` with the port defaulting to `9494`.

Server-Side Usage

Starting a Server

A server is started from an existing DuckDB session. Everything that the session can see (in-memory tables, attached files, schemas) becomes reachable over the remote protocol.

To start listening on localhost, run:

SERVER

```
CALL quack_serve('quack:localhost')
```

Installation

Documentation

Getting Started

Connect

Data Import and Export

Lakehouse Formats

Client APIs

SQL

Configuration

Extensions

Core Extensions

Quack Remote Protocol

Overview

Reference

Security

Setup

Troubleshooting

Guides

Operations Manual

Development

Internals

Sitemap

Live Demo

`quack_serve` returns the listen URI, the HTTP URL, and, when the default authentication function is in use, and the `auth_token` that clients need to connect. This token can also be set explicitly before starting (see [Security](#)).

By default the server refuses to bind anything other than a local hostname. To listen on an externally-reachable address, pass `allow_other_hostname => true`:

```
SERVER
CALL quack_serve('quack:0.0.0.0:9494', {allow_other_hostname => true})
```

When you do this you should front the server with a TLS-terminating reverse proxy. See [Securing Quack with a Reverse Proxy](#).

URI Format

Quack endpoints use the `quack:` URI scheme and `9494` as the default port. Some examples:

URI	Host	Port
<code>quack:localhost</code>	<code>localhost</code>	<code>949</code>
<code>quack://localhost</code>	<code>localhost</code>	<code>949</code>
<code>quack:myhost:9000</code>	<code>myhost</code>	<code>900</code>
<code>quack:127.0.0.1</code>	<code>127.0.0.1</code>	<code>949</code>
<code>quack:[::1]:1234</code>	<code>::1</code>	<code>123</code>

You can parse and validate a URI with the `quack_uri_parser(uri, ssl)` scalar function.

Installation

Documentation

Getting Started

Connect

Data Import and Export

Lakehouse Formats

Client APIs

SQL

Configuration

Extensions

Core Extensions

Quack Remote Protocol

Overview

Reference

Security

Setup

Troubleshooting

Guides

Operations Manual

Development

Internals

Sitemap

Live Demo

Stopping a Server

To stop a server, run:

SERVER

```
CALL quack_stop('quack:localhost', ;
```

Client-Side Usage

There are two ways to talk to a Quack server:

1. `quack_query(uri, query)`: stateless querying.
2. `ATTACH 'quack:host' AS name`: attach the remote as a full catalog.

In both cases, you need to authenticate.

The client picks plain HTTP automatically for local URIs (`localhost`, `127.0.0.1`, `:::1`) and HTTPS otherwise. Either default can be overridden with the `DISABLE_SSL` configuration option.

Stateless Queries with `quack_query`

You can run any SQL against a server without attaching it. To query a local database via HTTP, run:

CLIENT

```
FROM quack_query(  
  'quack:localhost',  
  'SELECT 42',  
  token = 'MY_QUACK_TOKEN_0123456'
```

Remote databases use HTTPS by default. To override on a plain-HTTP remote, run:

CLIENT

Installation

Documentation

Getting Started

Connect

Data Import and Export

Lakehouse Formats

Client APIs

SQL

Configuration

Extensions

Core Extensions

Quack Remote Protocol

Overview

Reference

Security

Setup

Troubleshooting

Guides

Operations Manual

Development

Internals

Sitemap

Live Demo

```
FROM quack_query(  
  'quack:remote.com',  
  'SELECT 42',  
  token = 'MY_QUACK_TOKEN_012345678',  
  disable_ssl => true  
);
```

The query executes remotely and the server stream the result back. Errors occurring on the server (parse errors, missing tables, etc.) are shown locally in the DuckDB client.

Attaching a Remote Database

To attach a local database via HTTP, simply run:

CLIENT

```
ATTACH 'quack:localhost' AS remote_db  
TOKEN 'MY_QUACK_TOKEN_012345678'  
);
```

Attaching remote databases uses HTTPS by default. To override on a plain-HTTP remote:

CLIENT

```
ATTACH 'quack:remote.com' AS remote_db  
TOKEN 'MY_QUACK_TOKEN_012345678'  
DISABLE_SSL true  
);
```

Once attached, remote tables look and behave like local ones:

CLIENT

```
CREATE TABLE remote_db.t AS FROM remote_db  
INSERT INTO remote_db.t VALUES (42)
```

Installation

Documentation

Getting Started

Connect

Data Import and Export

Lakehouse Formats

Client APIs

SQL

Configuration

Extensions

Core Extensions

Quack Remote Protocol

Overview

Reference

Security

Setup

Troubleshooting

Guides

Operations Manual

Development

Internals

Sitemap

Live Demo

You can run queries against the remote database:

CLIENT

```
FROM remote_db.t;
FROM remote_db.t WHERE i = 42; -- r
BEGIN; ...; COMMIT;           -- t
DETACH quack;                  -- c
```

The attached catalog also exposes a `query` table macro for ad-hoc SQL scoped to that attachment:

CLIENT

```
FROM remote_db.query('SELECT 4. ,
```

Authentication

Clients pass the authentication token to the server in one of two ways: a `quack` secret scoped to the server URI or an explicit `TOKEN` option on `ATTACH` / `quack_query`. See [Security](#) for the full picture.

We recommend using a secret scoped to the server URI:

CLIENT

```
CREATE SECRET (
  TYPE quack,
  TOKEN 'MY_QUACK_TOKEN_012345678
  SCOPE 'quack:localhost'
);

ATTACH 'quack:localhost' AS remote_
```

Alternatively, you can pass the token directly, which overrides any matching secret:

CLIENT

HTTP Connection Caching

By default, each Quack client request opens a fresh connection to the server – a new TCP and, with SSL, a costly TLS handshake. Connection caching reuses connections across requests, reducing per-query latency on repeated requests:

```
SET httpfs_connection_caching
```

Node Identity (whoami)

Each Quack node exposes a `whoami()` table macro that surfaces basic identity and runtime info, useful when proxying to a fleet of servers or when correlating logs:

```
FROM remote_db.query('FROM who,')
```

name varchar	provider varchar	hostname varchar	...
NULL	NULL	NULL	...

Identity fields are populated either by setting `whoami_*` options directly or by calling the `quack_identify` helper:

CLIENT

Installation

Documentation

Getting Started

Connect

Data Import and Export

Lakehouse Formats

Client APIs

SQL

Configuration

Extensions

Core Extensions

Quack Remote Protocol

Overview

Reference

Security

Setup

Troubleshooting

Guides

Operations Manual

Development

Internals

Sitemap

Live Demo

```
CALL quack_identify(  
  name => 'analytics-1',  
  provider => 'ec2',  
  region => 'eu-west-1',  
  meta => '{"role": "worker"}'  
);
```

`meta` is merged with auto-computed `duckdb_version` and `platform` keys; user-supplied keys win on conflict. `whoami_started_at` (an ISO 8601 timestamp) overrides the uptime anchor, otherwise uptime is measured from extension load.

Pages in This Section

[Reference](#)

[Security](#)

[Setup](#)

[Overview](#)

[Reverse Proxy](#)

[Deployment](#)

[Using from Wasm](#)

[Troubleshooting](#)

 [See this page as Markdown](#)

 [Edit this page on GitHub](#)

 [Report content issue](#)

 [See this repository on GitHub](#)

Installation

Documentation

[Getting Started](#)

[Connect](#)

[Data Import and Export](#)

[Lakehouse Formats](#)

[Client APIs](#)

[SQL](#)

[Configuration](#)

[Extensions](#)

[Core Extensions](#)

[Quack Remote Protocol](#)

[Overview](#)

[Reference](#)

[Security](#)

[Setup](#)

[Troubleshooting](#)

[Guides](#)

[Operations Manual](#)

[Development](#)

[Internals](#)

[Sitemap](#)

[Live Demo](#)